



PROJECT-BASED LEARNING

PHASE-I EVALUATION

**MiniGit: A Lightweight Version Control
System Using C++ and Data Structures**

Team ID: DSCPP-III-2026-T296 | Team Name: Blaze

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026-27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T296
Team Name: Blaze
Section: *ML-1, ML-2, ML-3, ML-4*
Domain: CSE (AI&ML)

Team Members

1. Gangwar, Abhinesh – 2510370192 (Lead)
2. Jain, Sparsh – 2510370159
3. Bajaj, Krish – 2510370469
4. Joshi, Snehil – 2510370146

Mentor

Mr. Himanshu Mainwal

Interactions completed: _____

Problem Statement & Motivation

Problem Statement

Students repeatedly duplicate project folders (*final*, *final_new*, *final_latest*) to track progress, making it hard to identify the correct version, compare changes, or recover work after a mistake – a gap Git addresses, but with complexity that overwhelms beginners.

Motivation

- ▶ Manual folder copies are confusing and error-prone
- ▶ No simple way to compare or restore earlier work
- ▶ Professional VCS tools (Git, SVN) have a steep learning curve

Target Users / Stakeholders

- ▶ Beginners and students learning version control
- ▶ Individuals working on small local projects
- ▶ Benefit: a simple, transparent view of core VCS concepts

Objectives

Project Objectives

1. Build a lightweight local version-control tool (MiniGit) in C++.
2. Implement core commands: `init`, `add`, `commit`, `log`, `checkout`, `rollback`.
3. Apply OOP design together with linked list, hash table, and stack data structures in one integrated system.
4. Validate functionality through test cases and a sample repository demonstration.
5. Demonstrate a learning-focused alternative to professional VCS tools.

Key Objective: Deliver a working CLI tool that lets a user create a repository, track files, commit versions with messages, view history, and reliably restore any earlier version.

Proposed Solution

Proposed Approach

1. Input: terminal commands issued by the user
2. Processing: `CommandHandler` parses and validates arguments
3. System Logic: `Repository` manages tracking, commits, staging
4. Storage: `.minigit/` directory holds objects and metadata
5. Output: history log, restored files, status/error messages

Key Features

- ▶ `init` – create a local repository
- ▶ `add` – select files for tracking
- ▶ `commit` – save a version with ID, message, timestamp
- ▶ `log / checkout` – view and restore history
- ▶ `rollback` – undo the latest restoration

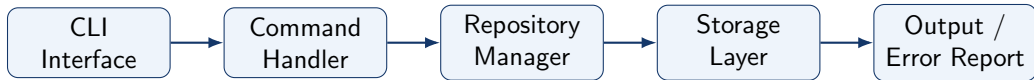
MiniGit distills version control to its essential workflow, keeping the core ideas easy to observe.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C++	Linked list (commit history), hash table (file/commit lookup), stack (rollback)	Repository & Restore Modules
OOPs with C++	Classes Repository, Commit, FileSnapshot, CommandHandler; encapsulation, modular design	Core Architecture
Operating Systems	File I/O, directory creation, and file-system management under .minigit/	Storage Layer
DBMS	Not used directly – flat-file metadata (HEAD, refs, index) replaces a database	Storage Layer

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow



Major Components

- ▶ Repository – init, tracking, commits, history
- ▶ Commit & FileSnapshot – metadata and snapshot storage
- ▶ CommandHandler – parses and validates CLI commands

Data / Control Flow

- ▶ Commands flow CLI → CommandHandler → Repository Manager
- ▶ Repository Manager updates the commit list, hash-table index, and undo stack
- ▶ Results persist in `.minigit/`; status echoes to the terminal

Technology Stack & Methodology

Technology Stack

- ▶ Programming: **C++**
- ▶ Storage: Flat files / custom objects under **.minigit/** (no external DB)
- ▶ Tools / Framework: Standard C++ file I/O, CLI argument parsing
- ▶ IDE: **VS Code**
- ▶ Version Control (team): Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Evaluation

C++ enables direct file-system control and applies OOP with data-structure concepts; flat-file storage keeps focus on core VCS logic over external systems.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Studied Git workflows
- ▶ Finalized command set: init, add, commit, log, checkout, rollback
- ▶ Planned `.minigit/` folder & metadata format

Individual Contribution

Member	Role	Contribution
Abhinesh	Team Lead	Architecture design; Repository Manager module; integration; report & demo
Sparsh	Developer	CLI Interface & Command Handler (parsing, dispatch, validation)
Krish	Developer	Data Structures: linked list, hash table, stack
Snehil	Developer/ Docs	Storage Layer (<code>.minigit/</code> object store, metadata); docs & testing

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Planning & Design
2. Phase-II: Repository & Commit Module
3. Phase-III: Restore, Test & Document

Expected Outcomes

- ▶ Working CLI prototype (init/add/commit/log/checkout/rollback)
- ▶ Verified commit history & restore correctness
- ▶ Demonstrates DS & OOP concepts together
- ▶ Optional: branching & file-diff, if time permits

References

1. Git SCM, *Git Documentation* – git-scm.com/doc
2. Chacon & Straub, *Pro Git*, 2nd Ed. – git-scm.com/book/en/v2
3. cppreference.com, *C++ Standard Library Reference* – en.cppreference.com
4. Pat Morin, *Open Data Structures (in C++)* – opendatastructures.org/ods-cpp

Thank You
Questions & Suggestions